

Analysis: Curling

Let us denote the score of the red team and the yellow team as s_{red} and s_{yellow} , respectively.

Test Set 1

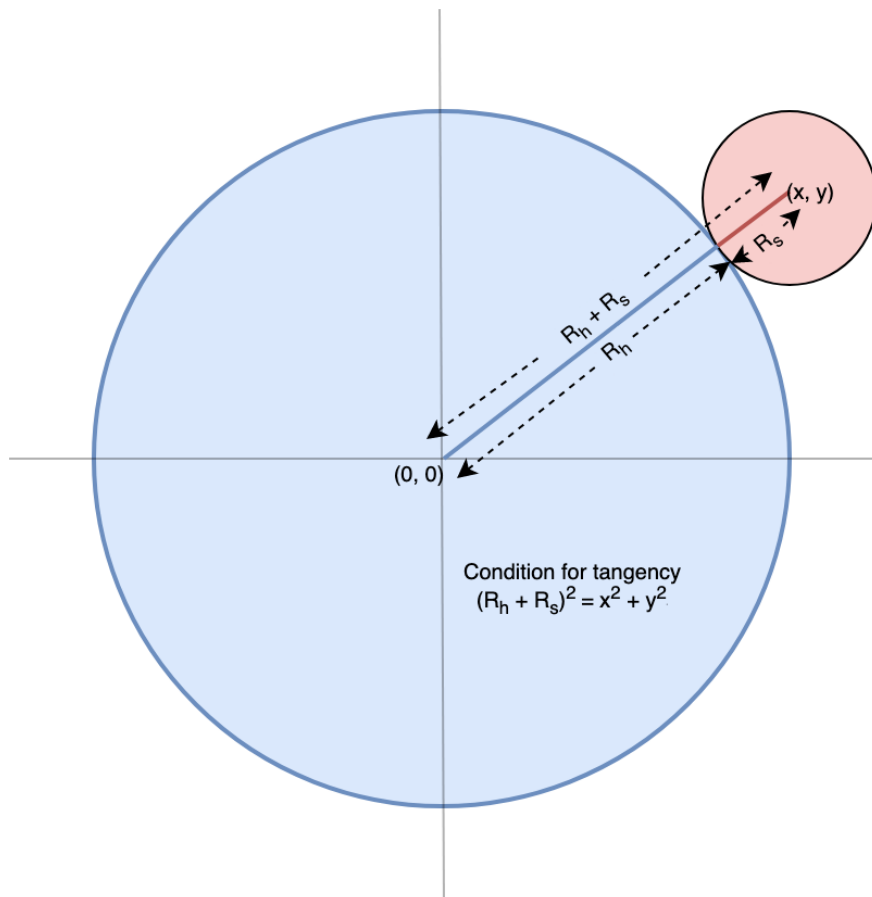
For this test set, $M = 0$, i.e., there are no stones remaining on the curling sheet for the yellow team. In this case:

- s_{red} = number of stones which are in the house.
- $s_{yellow} = 0$.

For a house of radius R_h centered at $(0, 0)$, a stone centered at (x, y) with radius R_s is:

1. in the house iff: $\sqrt{x^2 + y^2} \leq R_h + R_s$ (Equality is for the case when the stone and the house are tangent to each other. Figure 1 shows such a case).
2. outside the house iff: $\sqrt{x^2 + y^2} > R_h + R_s$.

To count the stones in the house, we can iterate over the stones and count those which satisfy condition 1, i.e., $x^2 + y^2 \leq (R_h + R_s)^2$.



Here is a sample code in C++:

```
int s_red = 0, s_yellow = 0;
for(int i = 1; i <= N; i++) {
    s_red += (x[i] * x[i] + y[i] * y[i]) <= (rh + rs) * (rh + rs);
}
```

Test Set 2

In this test set, we can have non-zero number of stones remaining for both teams. Figure 2 shows some examples of scoring in the game. To calculate the score of a team, we can count the number of stones which contribute to the score of a team. A stone contributes to the score of a team iff:

1. It is in the house.
2. It is closer to the center (0, 0) than all of the stones of the opponent team.

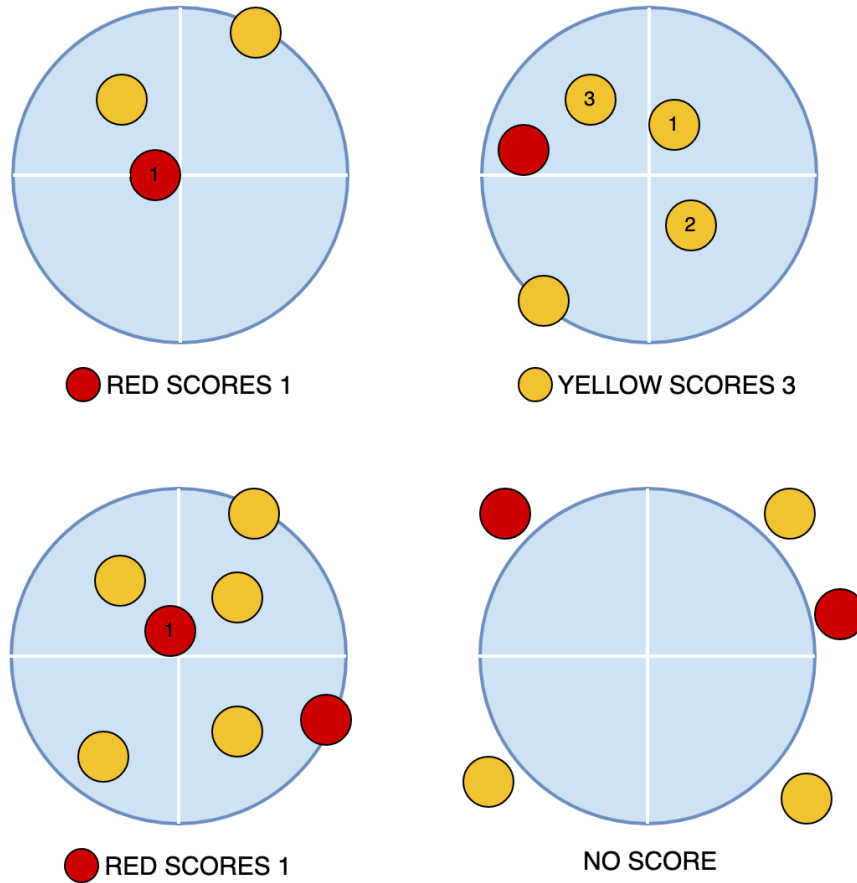


Figure 2: Examples of scoring in curling. Numbered circles are the only scoring ones.

Here is a sample code in C++:

```
int dist(int x, int y) { return x * x + y * y; }

void solve() {
    int s_red = 0;
    for(int i = 1; i <= N; i++) {
        bool is_scoring = dist(x[i], y[i]) <= (rs + rh) * (rs + rh); // Inside house.
        for(int j = 1; j <= M; j++) {
            is_scoring &= dist(x[i], y[i]) < dist(z[j], w[j]);
        }
        s_red += is_scoring;
    }

    int s_yellow = 0;
    for(int i = 1; i <= M; i++) {
        bool is_scoring = dist(z[i], w[i]) <= (rs + rh) * (rs + rh); // Inside house.
        for(int j = 1; j <= N; j++) {
            is_scoring &= dist(z[i], w[i]) < dist(x[j], y[j]);
        }
        s_yellow += is_scoring;
    }
}
```

The overall time complexity of the above solution would be $O(N \times M)$.

Another solution

Note that the score of at least one team must be 0. If a team does not have any stones on the curling sheet, their score is 0. If both teams have at least one stone still in play, the opponent of the team that has the stone closest to the center will have a 0 score.

- Case 1: $N = 0$ or $M = 0$, i.e. there is at least one team that does not have any stones remaining on the curling sheet.
 - In this case, the score of the team which does not have any stones left is 0 and the score of the other team is the number of stones in the house.
- Case 2: $N > 0$ and $M > 0$, i.e. each team has at least one stone on the curling sheet.

- Let m_{red} be the least squared distance of a stone of the red team, and m_{yellow} be the least squared distance of a stone of the yellow team.

$$m_{red} = \min_{1 \leq i \leq N} (\mathbf{X}_i^2 + \mathbf{Y}_i^2)$$

$$m_{yellow} = \min_{1 \leq i \leq M} (\mathbf{Z}_i^2 + \mathbf{W}_i^2)$$

Note: $m_{red} \neq m_{yellow}$, as no two stones can be equally close to the center $(0, 0)$.

The score of teams:

$$s_{red} = \begin{cases} \text{number of stones such that } \mathbf{X}_i^2 + \mathbf{Y}_i^2 < m_{yellow} \text{ and } \mathbf{X}_i^2 + \mathbf{Y}_i^2 \leq (\mathbf{R}_h + \mathbf{R}_s)^2, \\ 0, \end{cases} \quad \begin{matrix} m_{red} < m_{yellow} \\ m_{red} > m_{yellow} \end{matrix}$$

$$s_{yellow} = \begin{cases} \text{number of stones such that } \mathbf{Z}_i^2 + \mathbf{W}_i^2 < m_{red} \text{ and } \mathbf{Z}_i^2 + \mathbf{W}_i^2 \leq (\mathbf{R}_h + \mathbf{R}_s)^2, \\ 0, \end{cases} \quad \begin{matrix} m_{yellow} < m_{red} \\ m_{yellow} > m_{red} \end{matrix}$$

The overall time complexity of the above solution would be $O(N + M)$.

Analysis: Cute Little Butterfly

Test Set 1

Let us introduce some notation first.

- The maximum x -coordinate of a flower $X_{\max} = \max_{1 \leq i \leq N} X_i$.
- The maximum y -coordinate of a flower $Y_{\max} = \max_{1 \leq i \leq N} Y_i$.

For $N \leq 6$, we can enumerate all valid sequences of flowers and take the sequence yielding the highest energy. A sequence of flowers is valid if their y -coordinates form a non-increasing sequence and thus do not require any upward moves, which are illegal. As we visit the flowers one by one, there is no incentive of changing the direction if the next flower can be reached without doing so. This leads to the following recursive method using Python-like syntax:

```
used = [False] * N
ans = 0
enumerate(x=0, y=Y_max+1, is_right=True, energy=0)
print(ans)

def enumerate(x, y, is_right, energy):
    ans = max(ans, energy)
    for i in range(N):
        if not used[i] and Y[i] <= y:
            used[i] = True
            if (is_right and X[i] < x) or (not is_right and X[i] > x):
                # Need to change direction to reach the i-th flower
                enumerate(X[i], Y[i], not is_right, energy + C[i] - E)
            else:
                # Can reach the i-th flower without changing the direction
                enumerate(X[i], Y[i], is_right, energy + C[i])
            used[i] = False
```

The time complexity of this brute-force method is $O(N!)$.

Test Set 2

Let us define another function.

- The energy function $c(x, y) = \begin{cases} C_i, & \text{if } (x, y) = (X_i, Y_i) \text{ for some flower } i, \\ 0, & \text{otherwise.} \end{cases}$

The following observation will simplify our reasoning about the problem. Since going straight without changing the direction does not consume any energy, we can assume that direction changes happen at the x -coordinates 0 and $X_{\max}$ only. This, in turn, implies the existence of an optimal path where the flowers at the same y -coordinate are visited in one straight movement from left to right or from right to left without changing the direction in between. Therefore, we can clearly distinguish between the two scenarios when the flowers at level y are visited in the right or left direction.

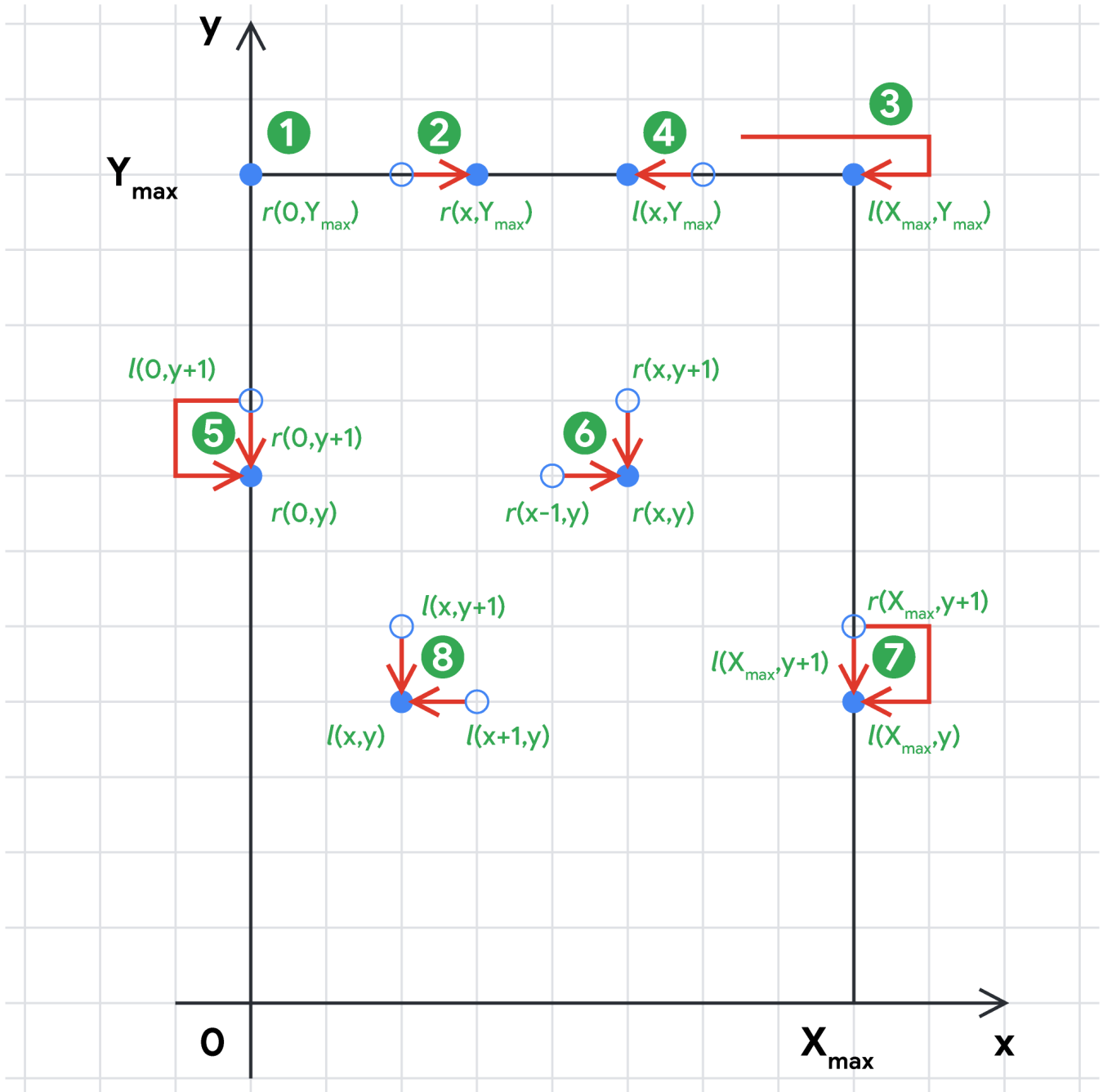
Let $r(x, y)$ be the maximum energy of a path ending at the point (x, y) when the flowers at level y are visited in the right direction, and let us define a similar function $l(x, y)$ for the left direction.

The final answer is then $\max_{0 \leq x \leq X_{\max}, 0 \leq y \leq Y_{\max}} \max(r(x, y), l(x, y))$.

The values of $r(x, y)$ and $l(x, y)$ can be computed using dynamic programming and the following recurrence relations. As the relations suggest, we should calculate the functions $r(x, y)$ and $l(x, y)$ for higher values of y first. For points in the same y level, we calculate $r(x, y)$ from left to right, and $l(x, y)$ from right to left.

1. $r(0, Y_{\max}) = c(0, Y_{\max})$,
2. $r(x, Y_{\max}) = r(x - 1, Y_{\max}) + c(x, Y_{\max})$ for $x > 0$,
3. $l(X_{\max}, Y_{\max}) = c(X_{\max}, Y_{\max}) - \mathbf{E}$,
4. $l(x, Y_{\max}) = l(x + 1, Y_{\max}) + c(x, Y_{\max})$ for $x < X_{\max}$,
5. $r(0, y) = \max[r(0, y + 1), l(0, y + 1) - \mathbf{E}] + c(0, y)$ for $y < Y_{\max}$,
6. $r(x, y) = \max[r(x - 1, y), r(x, y + 1)] + c(x, y)$ for $x > 0$ and $y < Y_{\max}$,
7. $l(X_{\max}, y) = \max[l(X_{\max}, y + 1), r(X_{\max}, y + 1) - \mathbf{E}] + c(X_{\max}, y)$ for $y < Y_{\max}$,
8. $l(x, y) = \max[l(x + 1, y), l(x, y + 1)] + c(x, y)$ for $x < X_{\max}$ and $y < Y_{\max}$.

The following image illustrates these recurrence relations.



The time complexity of such a dynamic programming solution is $O(X_{\max} \times Y_{\max})$, which is efficient enough as the coordinates are bounded by 500.

Test Set 3

For the large test set, the coordinate space is not reasonably restricted, so we should confine the calculation of $r(x, y)$ and $l(x, y)$ to the set of points with flowers. The general dynamic programming idea remains the same, though.

Assuming that we are currently processing the flower i , let us consider the recurrence relation (6). We can reach the i -th flower from any processed flower j moving in the right direction if $X_j \leq X_i$. Among all such flowers j , we are looking for the one with the largest $r(X_j, Y_j)$. To make such a lookup efficient, let us store the processed flowers in a set S_r sorted by X_j . Moreover, if $X_j \leq X_k$ for two processed flowers $j \neq k$ and $r(X_j, Y_j) \geq r(X_k, Y_k)$, we can safely ignore the flower k and drop it from S_r , so the set is essentially increasing by values $r(X_j, Y_j)$ as well. Now, to find the best processed flower j to visit the current flower i from, we just look for the flower j in S_r with the highest X_j such that $X_j \leq X_i$. Once we are done calculating $r(X_i, Y_i)$, we add the flower i to the set S_r and potentially drop some other flowers to maintain the sorted property of the set.

In order to facilitate the calculation of $l(x, y)$, we should maintain a similar set of processed flowers S_l , which is increasing by the coordinates X_j and decreasing by the values $l(X_j, Y_j)$.

Equipped with the sets S_r and S_l , we can also handle the border cases like items (5) and (7) above. Suppose we have calculated the functions r and l for all flowers i with $Y_i > y$ for some level y and we are about to process the flowers i with $Y_i = y$. Let a be the first (i.e. leftmost) flower in S_l and b be the last (i.e. rightmost) flower in S_r . To account for the change of direction before visiting the leftmost flower i with $Y_i = y$, we should make sure $r(X_i, Y_i)$ is at least $l(X_a, Y_a) + C_i - E$ in analogy with the recurrence relation (5) above. Similarly, for the rightmost flower i with $Y_i = y$, $l(X_i, Y_i)$ is at least $r(X_b, Y_b) + C_i - E$ in analogy with the recurrence relation (7).

The time complexity of this modified dynamic programming approach is $O(N \log N)$, as it involves sorting the flowers by their coordinates and using a standard sorted set data structure with $O(\log N)$ time per operation.

Analysis: Happy Subarrays

For simplicity, let us denote subarray of array $\mathbf{A}$ starting from index i and ending at index j , ($j \geq i$) as $A(i, j)$.

Test Set 1

$A(i, j)$ is a happy subarray iff all of its prefix sums are non-negative, i.e.

$$\begin{aligned} \mathbf{A}_i &\geq 0 \\ \mathbf{A}_i + \mathbf{A}_{i+1} &\geq 0 \\ \mathbf{A}_i + \mathbf{A}_{i+1} + \mathbf{A}_{i+2} &\geq 0 \\ &\vdots \\ \mathbf{A}_i + \mathbf{A}_{i+1} + \mathbf{A}_{i+2} + \cdots + \mathbf{A}_j &\geq 0 \end{aligned}$$

We can observe that:

- Observation 1: If $A(i, j)$ is a happy subarray then all its prefix arrays $A(i, k)$, such that $i \leq k \leq j$ are also happy subarray.
- Observation 2: If $A(i, j)$ is *not* a happy subarray then all subarrays $A(i, k)$, such that $k \geq j$ are also *not* happy subarray.

We can iterate over all subarrays with a left index i . For a fixed left index i , we can iterate over the right index j such that the subarray sum is non-negative. As soon as we find an index j such that subarray sum of $A(i, j)$ is less than 0, we can stop and increase the left index.

Here is a sample code in C++.

```
int ans = 0;
for(int i = 1; i <= N; i++) {
    int cur_sum = 0;
    for(int j = i; j <= N; j++) {
        cur_sum += A[j];
        if(cur_sum < 0)
            break;
        ans += cur_sum;
    }
}
```

The overall time complexity of the above solution would be $O(N^2)$, which is fast enough for test set 1.

Test Set 2

Let us denote subarray sum of $A(i, j)$ as $S(i, j)$ and prefix sum of array $\mathbf{A}$ till i^{th} index as $P(i)$,

$$\begin{aligned} S(i, j) &= \mathbf{A}_i + \mathbf{A}_{i+1} + \mathbf{A}_{i+2} + \cdots + \mathbf{A}_j \\ P(i) &= \mathbf{A}_1 + \mathbf{A}_2 + \mathbf{A}_3 + \cdots + \mathbf{A}_i \end{aligned}$$

The prefix sum array P of array $\mathbf{A}$ can be computed in $O(\mathbf{N})$ by iterating over the array from left to right:

$$P(i) = \begin{cases} 0 & i = 0 \\ P(i-1) + \mathbf{A}_i & i > 0 \end{cases}$$

By the definition of a prefix array, we can easily note that $S(i, j) = P(j) - P(i-1)$

For every index i , let us compute $nsv(i)$ (nearest smaller value), the smallest index j such that $j \geq i$ and subarray sum of $A(i, j)$ is less than 0. If there is no such index we can simply set $nsv(i) = \mathbf{N} + 1$. Finding smallest index j on right of i , such that the subarray sum $A(i, j)$ is less than 0

$$\begin{aligned} \mathbf{A}_i + \mathbf{A}_{i+1} + \mathbf{A}_{i+2} + \dots + \mathbf{A}_j &< 0 \\ S(i, j) &< 0 \\ P(j) - P(i-1) &< 0 \\ P(j) &< P(i-1) \end{aligned}$$

is same as finding the smallest index j , $j \geq i$ and $P(j) < P(i-1)$. This can be done using small modification in [All nearest smaller values algorithm](#) in $O(\mathbf{N})$.

All subarrays which starts at index l and end at index j , such that $l \leq j < nsv(l)$ would have non-negative sum. Sum of all such subarrays starting at index l and extending at max to index r , $r = nsv(l) - 1$ is same as the sum of below expressions:

$$\begin{aligned} \mathbf{A}_l &= P(l) - P(l-1) \\ \mathbf{A}_l + \mathbf{A}_{l+1} &= P(l+1) - P(l-1) \\ \mathbf{A}_l + \mathbf{A}_{l+1} + \mathbf{A}_{l+2} &= P(l+2) - P(l-1) \\ &\vdots \\ \mathbf{A}_l + \mathbf{A}_{l+1} + \mathbf{A}_{l+2} + \dots + \mathbf{A}_r &= P(r) - P(l-1) \end{aligned}$$

On simplification, sum of all subarray $A(i, j)$ such that $i = l$ and $i \leq j \leq r$

$$sum(l) = (P(l) + P(l+1) + P(l+2) + \dots + P(r)) - (r - l + 1) \times P(l-1)$$

The first term $P(l) + P(l+1) + P(l+2) + \dots + P(r)$ can be computed by pre-calculating the prefix sum array of P .

Let us denote prefix sum of P as PP , i.e. $PP(i) = P(1) + P(2) + \dots + P(i)$. The above sum can be simplified as:

$$\begin{aligned} sum(l) &= PP(r) - PP(l-1) - (r - l + 1) \times P(l-1) \\ ans &= \sum_{l=1}^N sum(l) \\ ans &= \sum_{l=1}^N PP(r) - PP(l-1) - (r - l + 1) \times P(l-1) \end{aligned}$$

Nearest smaller value on right for each index in prefix array $nsv(i)$ can be computed in $O(\mathbf{N})$. Sum of all subarrays with fixed left index and moving right index can be computed in $O(1)$, if we have pre computed prefix sum array of $\mathbf{A}$ i.e. P and prefix sum array of P i.e. PP .

Precomputation of prefix sum arrays can be done in $O(\mathbf{N})$. The overall time complexity of the above solution would be $O(\mathbf{N})$

Analysis: Walktober

Test Set 1

We are given the daily steps count of only two people -- John, and another participant. To make sure that John had the maximum steps on each day, we just need to compare his steps on each day with the other participant's steps on that day. Let us maintain a count C of the steps John needs in order to achieve his goal. Before day 1, C is equal to 0. Then, for each day from the beginning of Walktober:

- If John's steps were less than those of the other participant, we add the difference between their steps to C as that is the number of additional steps John needed.
- If John's steps were greater than or equal to those of the other participant, we continue to the next day as John already had the maximum steps for that day.

After the above process, we end up with the total number of steps John required last year in variable C .

Time and Space Complexity: Iterating over all the days would take $O(N)$ time. Overall time taken would be of the order of $O(N)$, with $O(1)$ extra space.

Test Set 2

To help visualize the input, let us make a 2D grid of each person's steps count:

ID	Day 1	Day 2	...	Day N
1	$S_{1,1}$	$S_{1,2}$	...	$S_{1,N}$
2	$S_{2,1}$	$S_{2,2}$	...	$S_{2,N}$
...	...	...	...	...
M	$S_{M,1}$	$S_{M,2}$	...	$S_{M,N}$

To calculate the answer, we first need to calculate the maximum steps taken by a participant each day. This would simply be the maximum step count over all M participants in that day's column in the table above. Let us denote this value for day j with $maxOfDay(j)$, where $1 \leq j \leq N$. $maxOfDay(j)$ can be calculated by traversing the day column j and keeping a track of the maximum steps encountered so far.

$$maxOfDay(j) = \max(S_{1,j}, S_{2,j}, \dots, S_{M,j})$$

Again, we maintain a count C of the number of steps John would have needed to achieve his goal, starting with $C = 0$. Recall from the statement that P denotes John's ID. For day j , John would require C_j additional steps, where:

$$C_j = (maxOfDay(j) - S_{P,j})$$

The total number of steps John required to achieve his goal (and therefore, the answer) would then be:

$$C = (C_1 + C_2 + \cdots + C_N)$$

Time and Space Complexity: *maxOfDay()* runs in $O(\mathbf{M})$ time to find the maximum steps over $\mathbf{M}$ participants on a particular day. Calculating *maxOfDay(j)* for each j such that $1 \leq j \leq \mathbf{N}$ would take $O(\mathbf{M} \cdot \mathbf{N})$ time. Overall time taken would be of the order of $O(\mathbf{M} \cdot \mathbf{N})$, with $O(\mathbf{N})$ extra space to calculate and store $C_1, C_2, \dots, C_N$.